

Reto: Paginacion Inteligente + Mejora Funcional

Explicacion detallada: antes vs. despues con justificacion

1. Migracion de fetch a axios

Antes:

```
const response = await fetch(url_api);  
let data = await response.json();
```

Justificacion del antes:

fetch es nativo de JS, no requiere librerias externas, y es suficiente para peticiones simples si la API responde con status 200.

Despues:

```
const response = await axios.get(url);  
let data = response.data;
```

Justificacion del despues:

axios maneja automaticamente errores HTTP (4xx/5xx lanzan excepcion, mientras fetch no), no requiere .json(), y permite configurar timeouts. Se pidio explicitamente en los requisitos del reto.

2. Paginacion bidireccional (Anterior + Siguiente)

Antes:

```
function loadMore() {  
  getElementButton(document, 'get')  
}
```

Justificacion del antes:

El codigo base solo demostraba 'cargar mas' en una direccion. Simple para un ejemplo minimo, pero insuficiente para navegacion completa.

Despues:

```
function loadMore(direction){  
  getElementButton(document, 'get', null, direction);  
}
```

Justificacion del despues:

El requisito exige botones 'Anterior' y 'Siguiente'. Anadir direction permite que ambos botones reutilicen la misma funcion, diferenciando el comportamiento con 'next' o 'prev' sin duplicar logica.

3. Control de estado de botones (deshabilitar)

Antes:

```
// Solo se seteaban atributos data-next y data-prev.  
button.setAttribute("data-next", (info.next===null)?'':info.next)  
button.setAttribute("data-prev", (info.prev===null)?'':info.prev)
```

Justificacion del antes:

El codigo base no requería deshabilitar botones porque solo habia uno ('loadMore') y no existia el concepto de 'ultima pagina'.

Despues:

```
btnNext.disabled = (info.next == null);  
btnPrev.disabled = (info.prev == null);
```

Justificacion del despues:

El requisito exige deshabilitar segun la posicion en la paginacion. La API devuelve null en info.next (ultima pag.) y null en info.prev (primera pag.). Asignar disabled directamente es la solucion mas simple.

4. Persistencia del estado actual (page indicator)

Antes:

```
// No existia. No se mostraba la pagina actual.
```

Justificacion del antes:

El codigo base no requería indicador de pagina porque no habia paginacion multiple.

Despues:

```
const pageParam = new URL(info.next ?? info.prev).searchParams.get('page');  
const currentPage = Number(pageParam);  
const totalPages = info.pages;  
const page = info.next ? currentPage - 1 : totalPages;  
document.getElementById('pageIndicator').textContent = `Page ${page} of ${totalPages}`;
```

Justificacion del despues:

El requisito opcional pide mostrar la pagina actual. La API no la envia directamente, pero esta codificada en las URLs de next y prev. Extraer el query param 'page' y calcular con info.next ? currentPage - 1 : totalPages resuelve el problema sin contadores manuales.

5. Reemplazar contenido (no acumular)

Antes:

```
function renderHtml(data){  
  // No habia limpieza inicial  
  for(...) { element.innerHTML += `<li>...</li>`; }  
}
```

Justificacion del antes:

En el codigo base solo se cargaba una pagina una vez, acumular no era un problema. innerHTML += era funcional para ese caso.

Despues:

```
function renderHtml(data){  
  element.innerHTML = ''; // limpia antes  
  for(...) { element.innerHTML += `<li>...</li>`; }  
}
```

Justificacion del despues:

Con paginacion, al navegar a otra pagina los resultados anteriores deben desaparecer. `innerHTML = ''` al inicio limpia el contenedor garantizando que no se mezclen personajes de distintas paginas.

6. Filtro dinamico por genero

Antes:

```
// No existia. No habia filtro.
```

Justificacion del antes:

El codigo base era un ejemplo minimo sin funcionalidades extra.

Despues:

```
let characters = [];  
  
// En requestData:  
characters = data.results;  
  
function filterByGender(){  
  const gender = document.getElementById("filterGender").value;  
  if(gender == ""){  
    renderHtml(characters);  
  } else {  
    const filtered = characters.filter(c => c.gender === gender);  
    renderHtml(filtered);  
  }  
}
```

Justificacion del despues:

El requisito exige filtrar sin recargar la API. Guardar `data.results` en `characters` preserva los datos originales. `filter()` crea un nuevo array sin mutar el original, permitiendo restaurar la vista al limpiar el filtro. Al cambiar de pagina, `characters` se sobrescribe reiniciando el filtro automaticamente.

7. Evitar multiples requests simultaneos

Antes:

```
// No existia. No habia control de concurrencia.
```

Justificacion del antes:

En el codigo base solo habia una carga inicial, no habia riesgo de doble click.

Despues:

```
let isLoading = false;  
  
async function requestData(url) {  
  if (isLoading) return;  
  isLoading = true;  
  // ... peticion ...  
}
```

```
    isLoading = false;
}
```

Justificacion del despues:

El requisito exige evitar requests duplicados. La bandera isLoading actua como candado: mientras una peticion esta en curso, cualquier click adicional en Siguiente/Anterior es ignorado.

8. Loading spinner

Antes:

```
// No existia.
```

Justificacion del antes:

El codigo base no lo requería y era una funcionalidad extra.

Despues:

```
document.getElementById("loading").style.display = "flex";
// ... peticion ...
document.getElementById("loading").style.display = "none";
```

Justificacion del despues:

El requisito opcional pide feedback visual. Mostrar un spinner durante la peticion mejora la experiencia de usuario, especialmente en conexiones lentas.

9. Shadowing de variable (correccion)

Antes:

```
async function requestData(url_api) {
```

Justificacion del antes:

Usar el mismo nombre para el parametro y la constante global funciona, pero crea ambigüedad: dentro de la funcion, url_api se refiere al parametro, no a la global.

Despues:

```
async function requestData(url) {
```

Justificacion del despues:

Renombrar el parametro a url elimina la ambigüedad. La variable global url_api permanece accesible si se necesita.

10. Arrow functions

Antes:

```
await new Promise(function (resolve){
    setTimeout(resolve, 500);
});
```

Justificacion del antes:

function es la sintaxis tradicional. Funciona correctamente y es la que muchos principiantes conocen primero.

Despues:

```
await new Promise(resolve => setTimeout(resolve, 500));
```

Justificacion del despues:

Arrow function es mas concisa, elimina llaves y return implicito para funciones de una linea. En callbacks cortos mejora la legibilidad.

11. parseInt a Number + || a ??**Antes:**

```
const currentPage = parseInt(new URL(info.next || info.prev).searchParams.get("page"));
```

Justificacion del antes:

parseInt convierte string a entero; || toma el primer valor truthy.
parseInt("12abc") da 12 (no avisa caracteres extra). || trataria "" como falsy aunque no sea null.

Despues:

```
const pageParam = new URL(info.next ?? info.prev).searchParams.get("page");  
const currentPage = Number(pageParam);
```

Justificacion del despues:

Number() es mas estricto: Number("12abc") da NaN, detectando datos malformados. ?? solo reemplaza null/undefined, siendo mas preciso semanticamente.

12. function a arrow + == a === en filter**Antes:**

```
const filtered = characters.filter(function (character) {  
    return character.gender == gender;  
});
```

Justificacion del antes:

function con return explicito y == son validos. == permite coercion de tipos, pero puede dar falsos positivos con coercion inesperada.

Despues:

```
const filtered = characters.filter(c => c.gender === gender);
```

Justificacion del despues:

Arrow function elimina codigo boilerplate. === evita coercion de tipos, comparando valor y tipo, mas seguro y predecible.